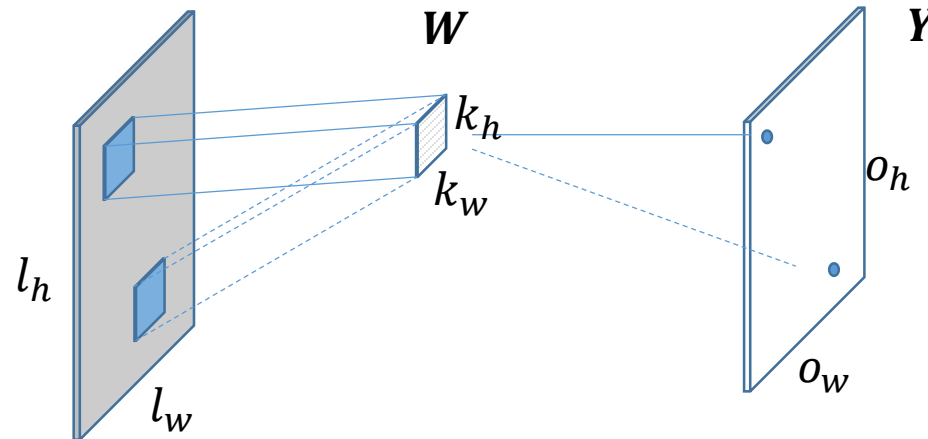


2D Convolutions

Application of convolution to images

2D Convolution

- 2D convolution follows the same principles, but the inputs, kernels and outputs are generalized into 2D matrices instead of 1D vectors

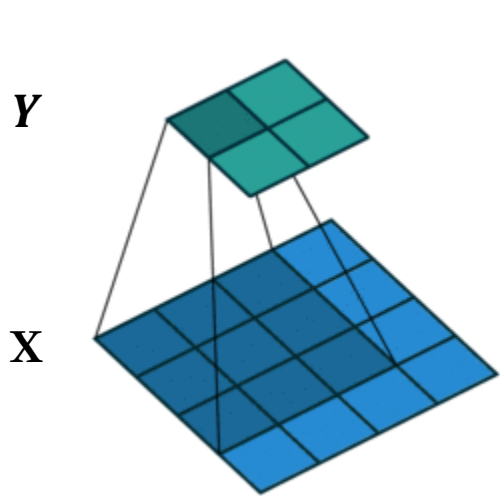


$$Y_{i,j} = \sum_{a=0}^{k_h-1} \sum_{b=0}^{k_w-1} X_{i*\underline{s_h}+a, j*\underline{s_w}+b} \times W_{a,b}$$

stride parameters in i & j

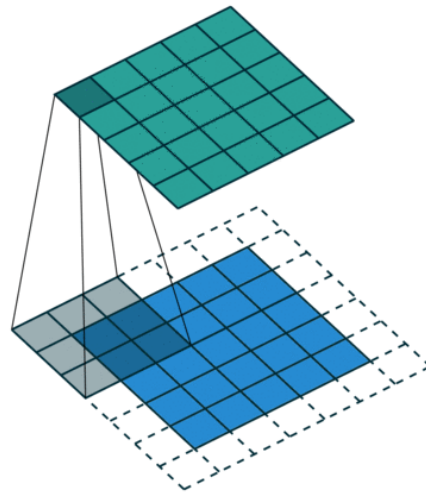
2D Convolution

“full” padding: ensures that every input element is used at all possible convolution locations, padding where necessary



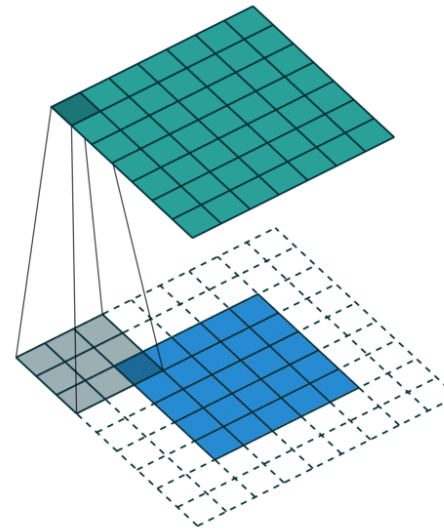
$k=3, p=0, s=1$ (Valid)

Kernel = 3
Padding = 0
Stride = 1



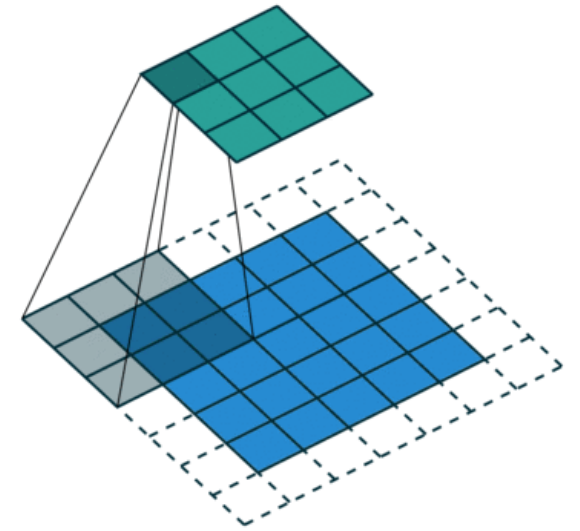
$k=3, p=2, s=1$ (Same)

Kernel = 3
Padding = 2
Stride = 1



$k=3, p=4, s=1$ (Full)

Kernel = 3
Padding = 4
Stride = 1



$k=3, p=2, s=2$

Kernel = 3
Padding = 2
Stride = 2

Colour Images



red channel

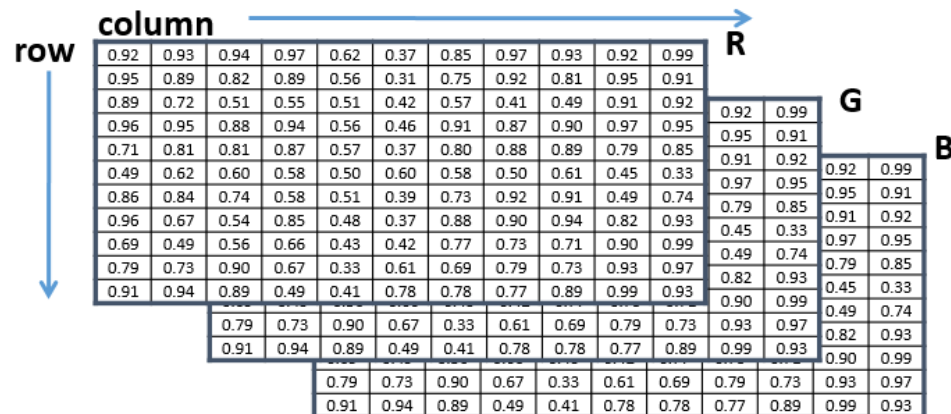


green channel



blue channel

$[0,0,0]$ is black, $[255,255,255]$ is white

[**R**, **G**, **B**] :[120, 30, 24]

represented as a matrix

$N \times M$ RGB image “im”

im(0,0,0) = top-left pixel value in R-channel

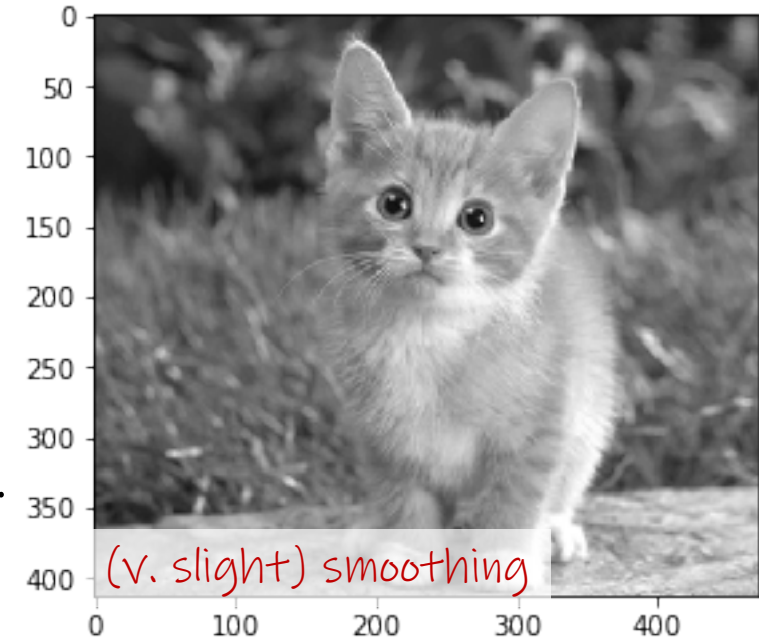
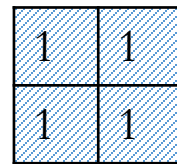
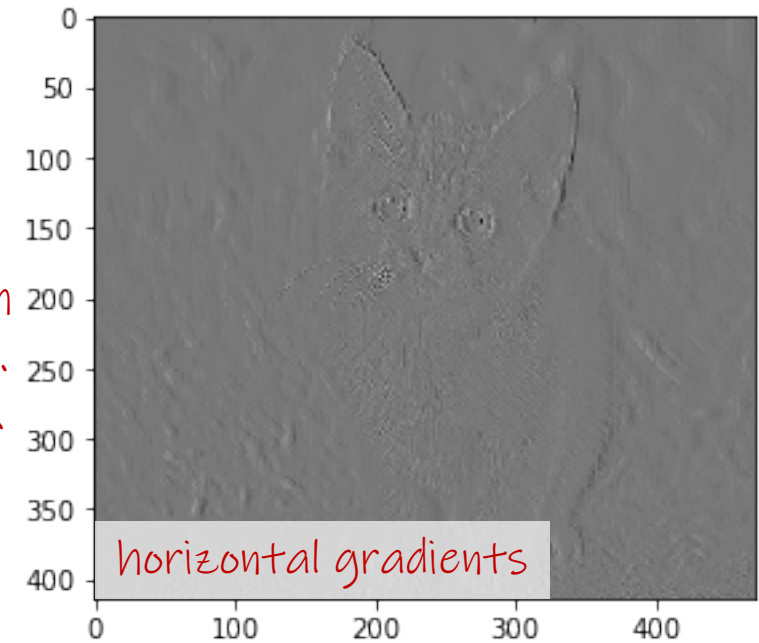
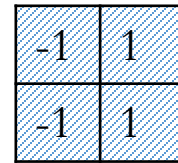
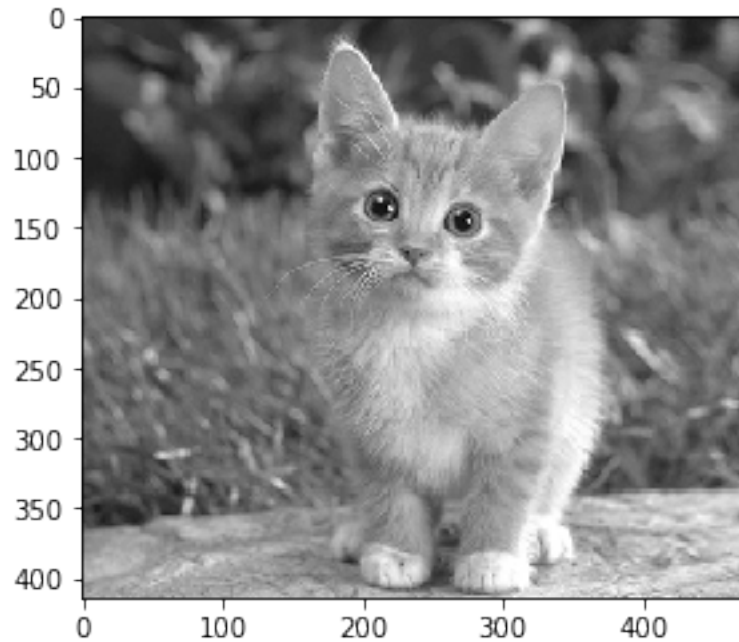
$\text{im}(y, x, b)$ = y pixels down, x pixels to right in the b^{th} channel

$$\text{im}(\mathbf{N}-1, \mathbf{M}-1, 2) = \text{bottom-right pixel in B-channel}$$

2D Convolution

In deep learning, we learn the kernels or weights which gives us good predictions e.g. classification, regression etc.

<http://setosa.io/ev/image-kernels/>



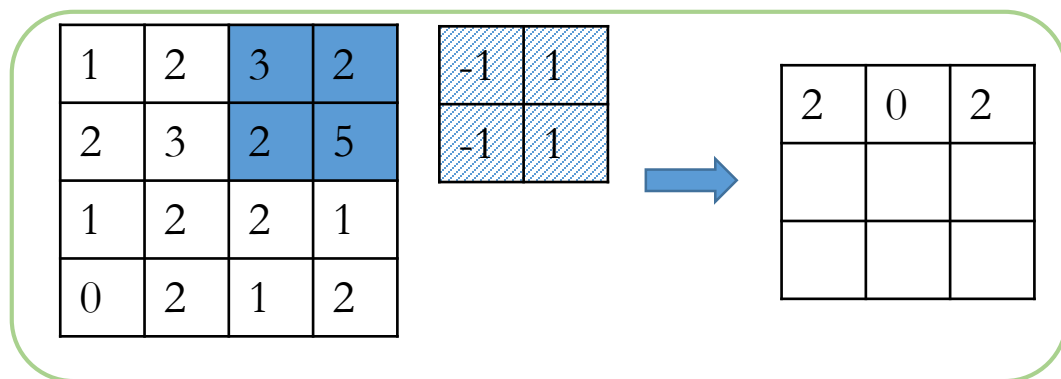
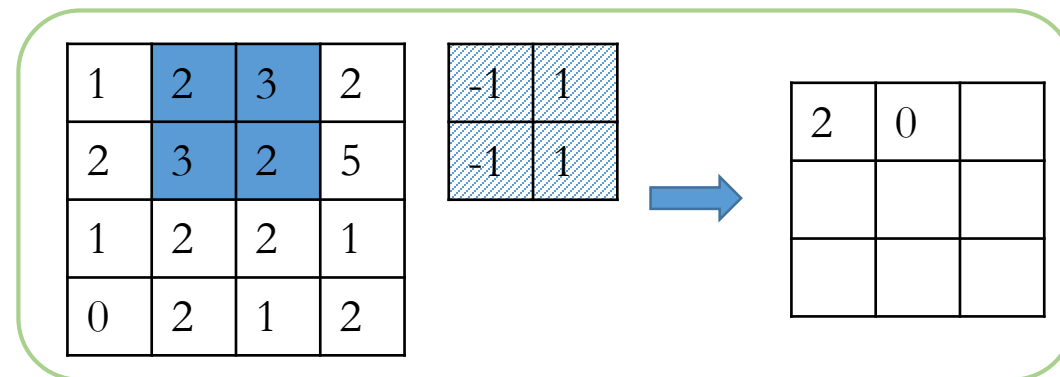
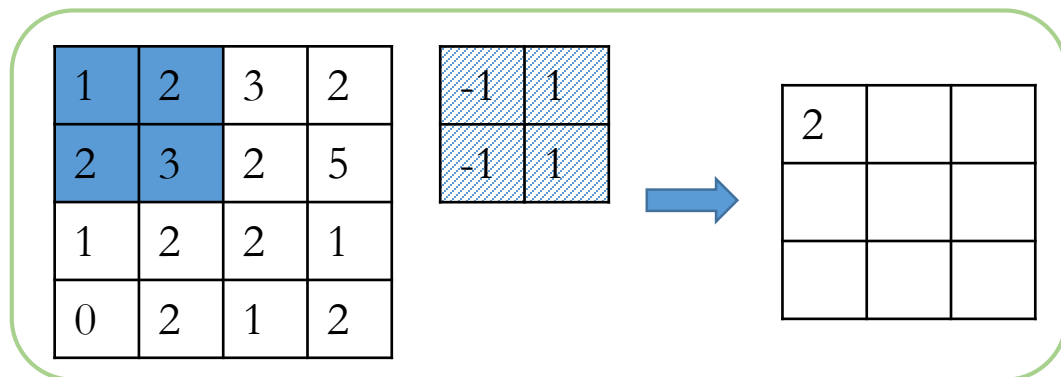
We get dif. feature maps depending on the kernel used.

2D Convolution

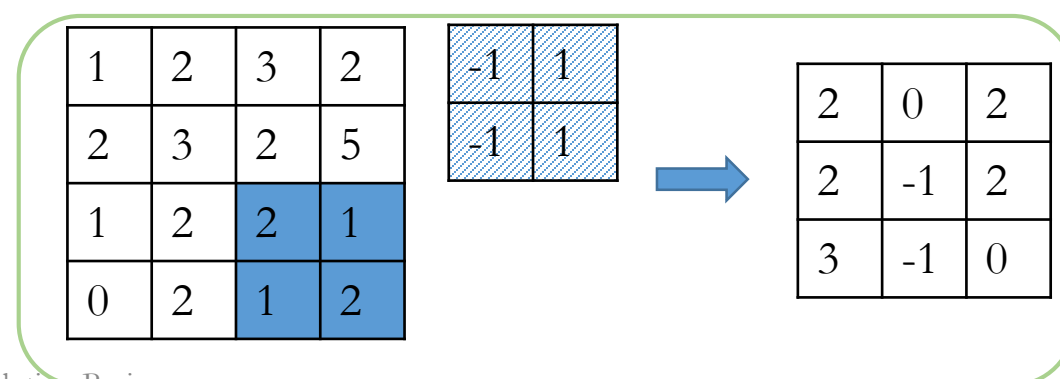
$$Y_{i,j} = \sum_{a=0}^{k_h-1} \sum_{b=0}^{k_w-1} X_{i*s_h+a, j*s_w+b} \times W_{a,b}$$



1	2	3	2
2	3	2	5
1	2	2	1
0	2	1	2



...



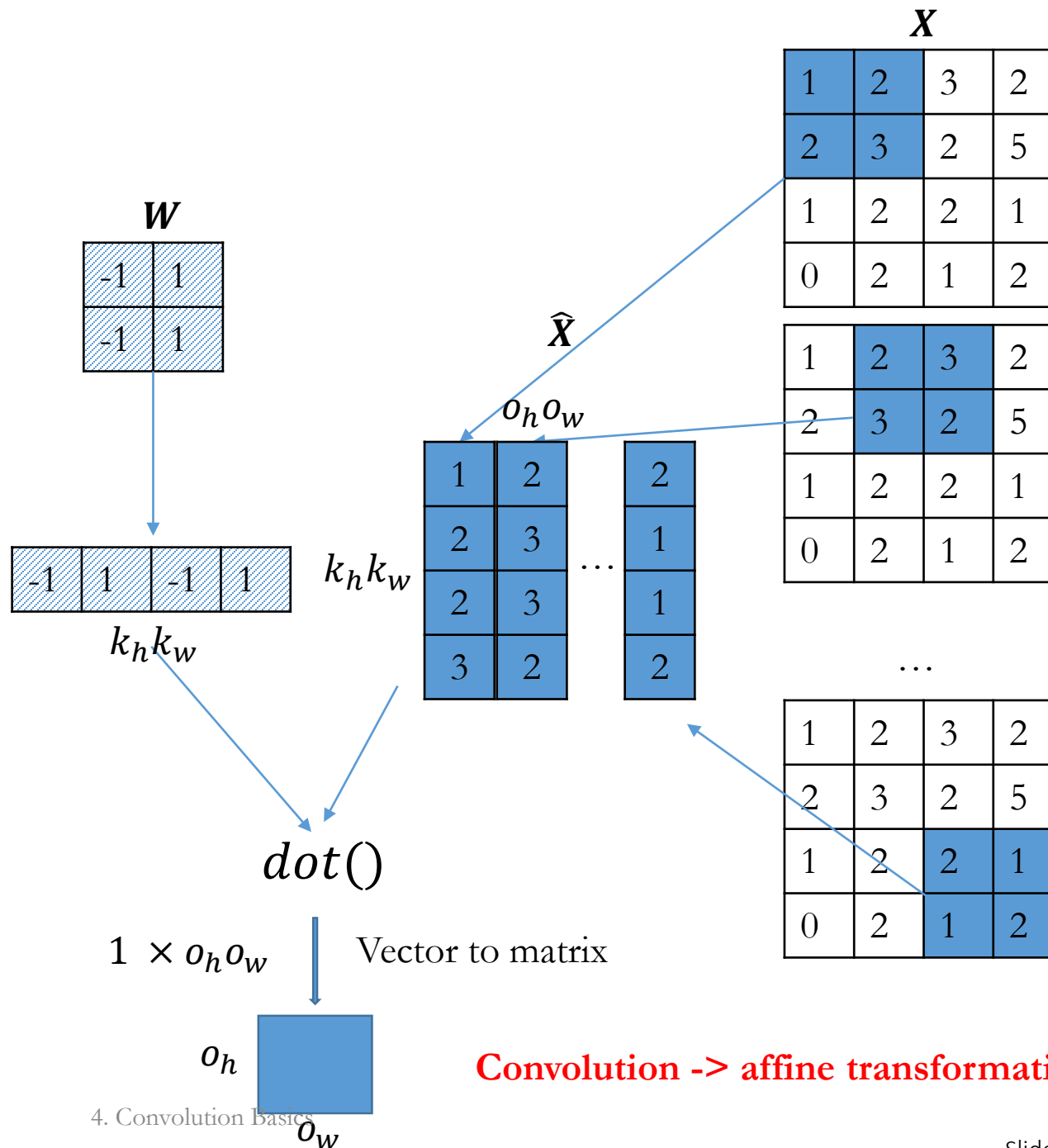
Implementation

$$Y_{i,j} = \sum_{a=0}^{k_h-1} \sum_{b=0}^{k_w-1} X_{i*s_h+a,j*s_w+b} \times W_{a,b}$$

This equation defines the result for a single element of Y , indexed by (i,j) . We've illustrated how to estimate the matrix of Y in series, i.e. one by one.

How can we estimate all of Y in vectorized form to perform the computations in parallel?

Img2Col: converts each receptive field into a column



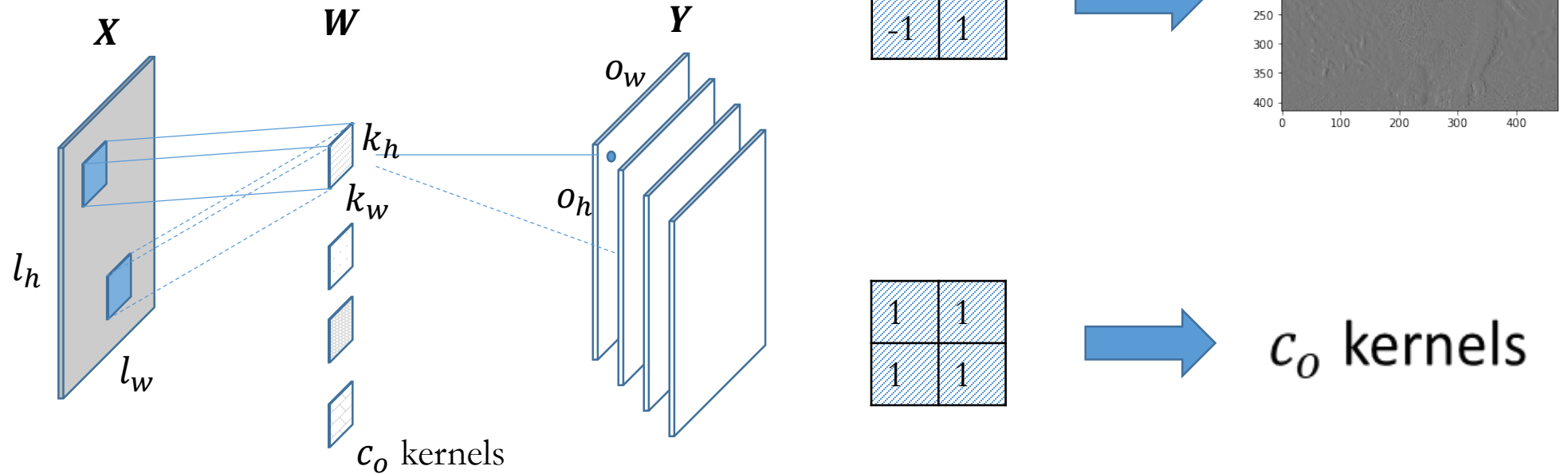
Convolution -> affine transformation

Statistics of Convolution

- Shape:
 - Input $\mathbf{X} \in R^{n_h \times n_w}$
 - Kernel $\mathbf{W} \in R^{k_h \times k_w}$
 - Output $\mathbf{Y} \in R^{o_h \times o_w}$
- Parameter size
 - $k_h \times k_w$
- Output shape
 - $(o_h, o_w) = (\left\lfloor \frac{n_h + p_h - k_h}{s_h} \right\rfloor + 1, \left\lfloor \frac{n_w + p_w - k_w}{s_w} \right\rfloor + 1)$
- Computation cost
 - $O(k_h \times k_w \times o_h \times o_w)$ (float multiplication ops, FLOP)

2D Convolution

- Multiple kernels/filters



single kernel definition

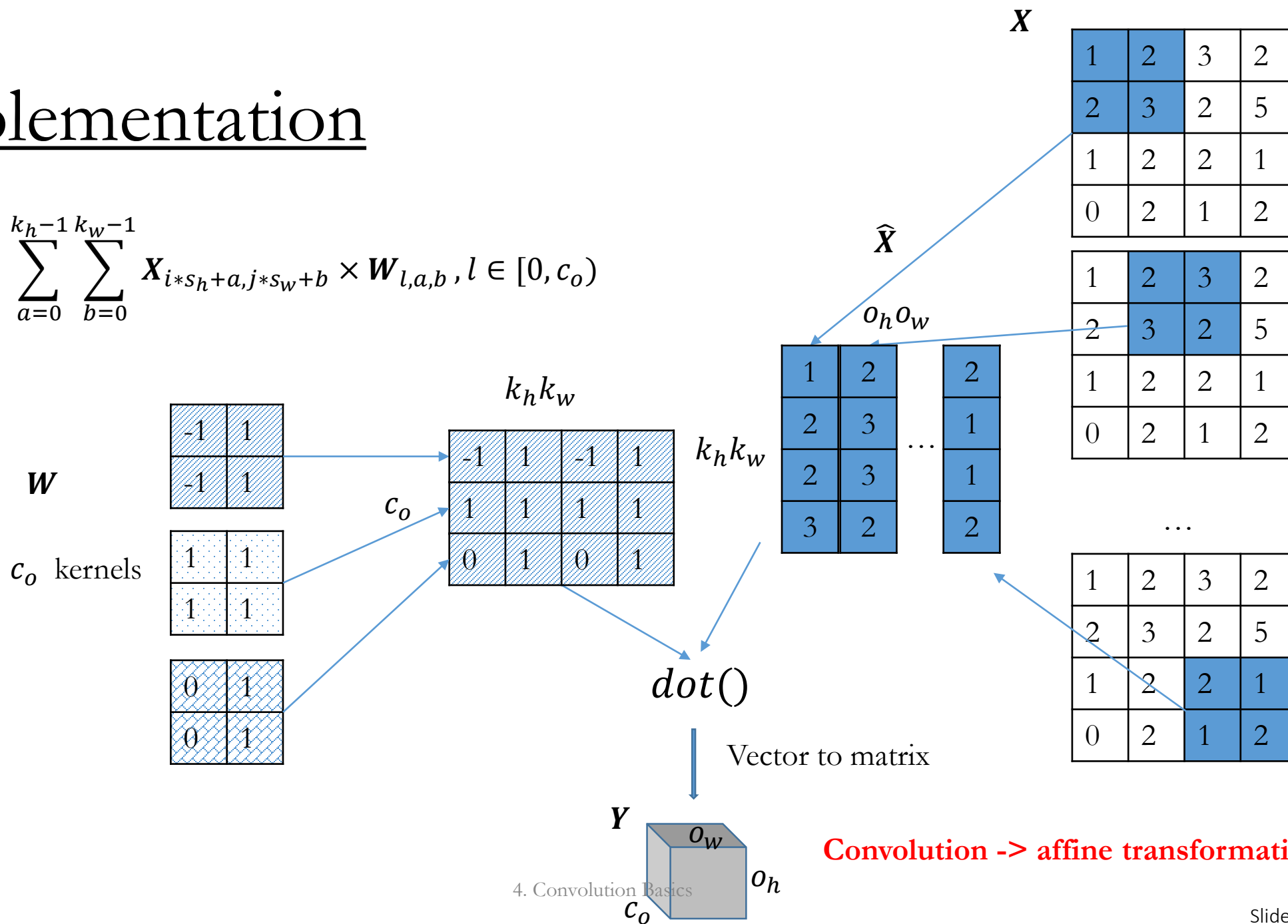
$$Y_{i,j} = \sum_{a=0}^{k_h-1} \sum_{b=0}^{k_w-1} X_{i*s_h+a, j*s_w+b} \times W_{a,b}$$

multiple kernel definition

$$Y_{l,i,j} = \sum_{a=0}^{k_h-1} \sum_{b=0}^{k_w-1} X_{i*s_h+a, j*s_w+b} \times W_{l,a,b}, l \in [0, c_o)$$

Implementation

$$Y_{l,i,j} = \sum_{a=0}^{k_h-1} \sum_{b=0}^{k_w-1} X_{i*s_h+a, j*s_w+b} \times W_{l,a,b}, l \in [0, c_o)$$

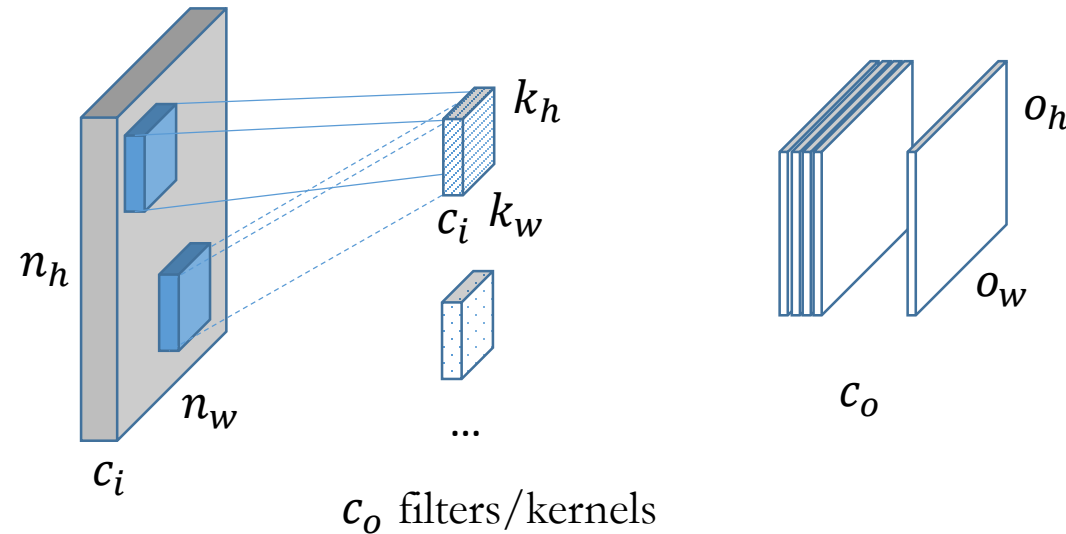


Statistics

- Applying multiple kernels (filters) c_o , all of the same stride and padding
- Parameter size
 - $c_o \times k_h \times k_w$
- Output shape
 - $(c_o, o_h, o_w) = (c_o, \left\lfloor \frac{n_h + p_h - k_h}{s_h} \right\rfloor + 1, \left\lfloor \frac{n_w + p_w - k_w}{s_w} \right\rfloor + 1)$
- Computation cost
 - $O((c_o \times k_h \times k_w) \times (o_h \times o_w))$ (float multiplication ops, FLOP)

2D Convolution

With multiple (c_i) input channels and kernels (filters)



*** The convolution results across all input channels are summed.*

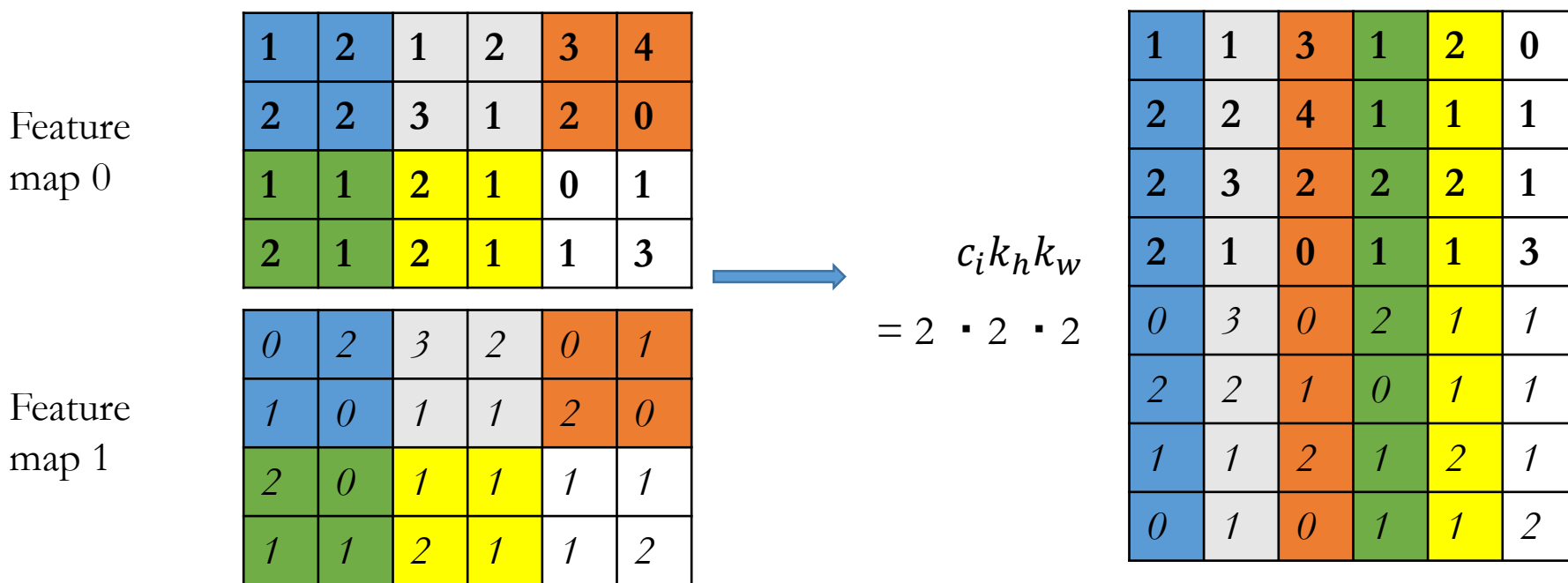
$$Y_{l,i,j} = \sum_{d=0}^{c_i-1} \underbrace{\sum_{a=0}^{k_h-1} \sum_{b=0}^{k_w-1} X_{d,i+a,j+b}}_{\text{summed across all input channels}} \times W_{l,d,a,b} + \mathbf{b}_l, l \in [0, c_o)$$

$\mathbf{b}$ is a bias vector

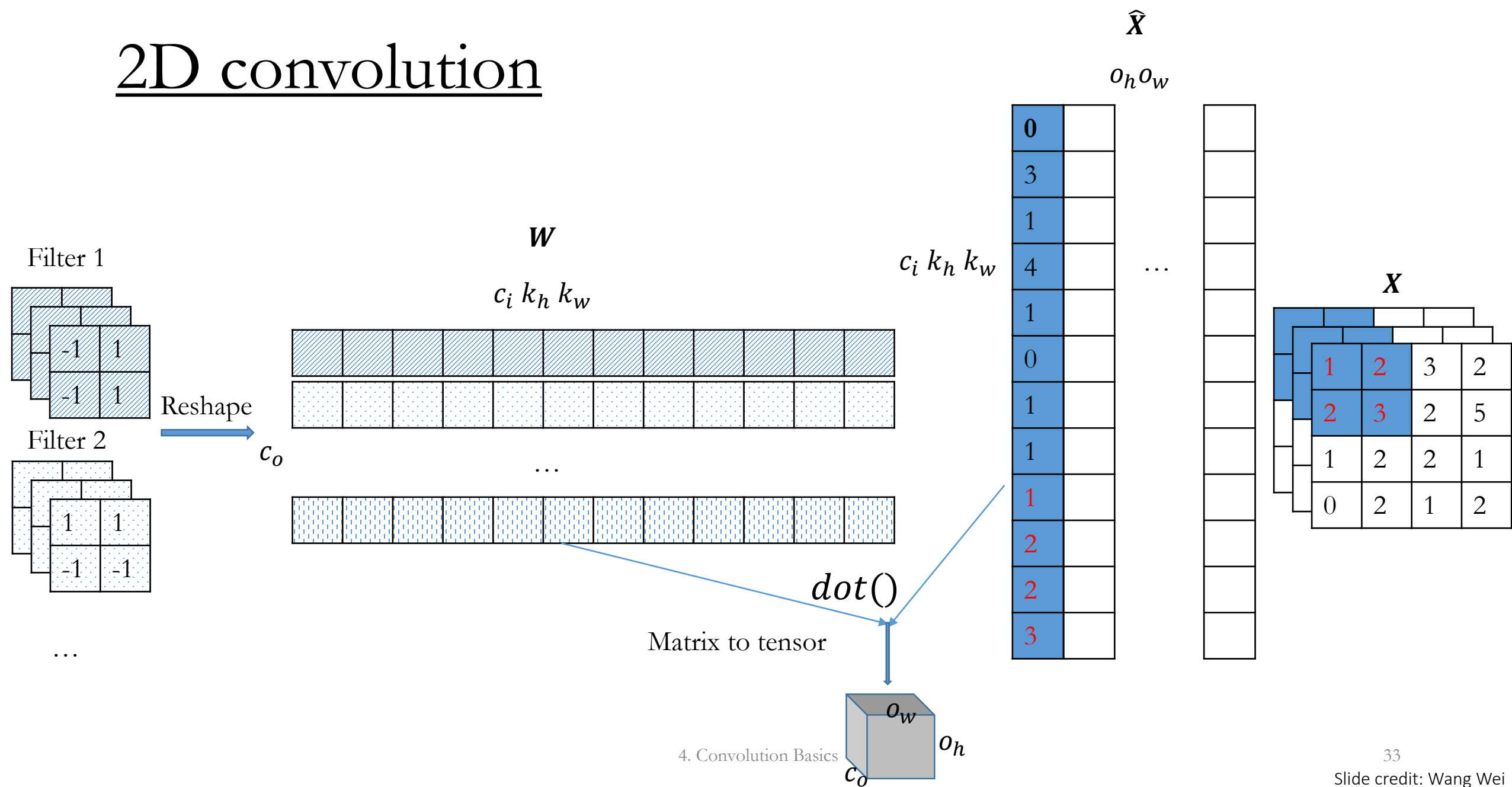
*** this is still a 2D convolution because the kernel is moved only across the horizontal and vertical dimensions (as indexed by a, b).*

Img2Col

- Slide the window from left to right, top to bottom
- Copy the values from the receptive field into a column of $\hat{\mathbf{X}}$
 - Receptive fields across feature maps are concatenated into to one column
- Reshape $\hat{\mathbf{X}}$ into $(c_i k_h k_w \times o_h o_w)$



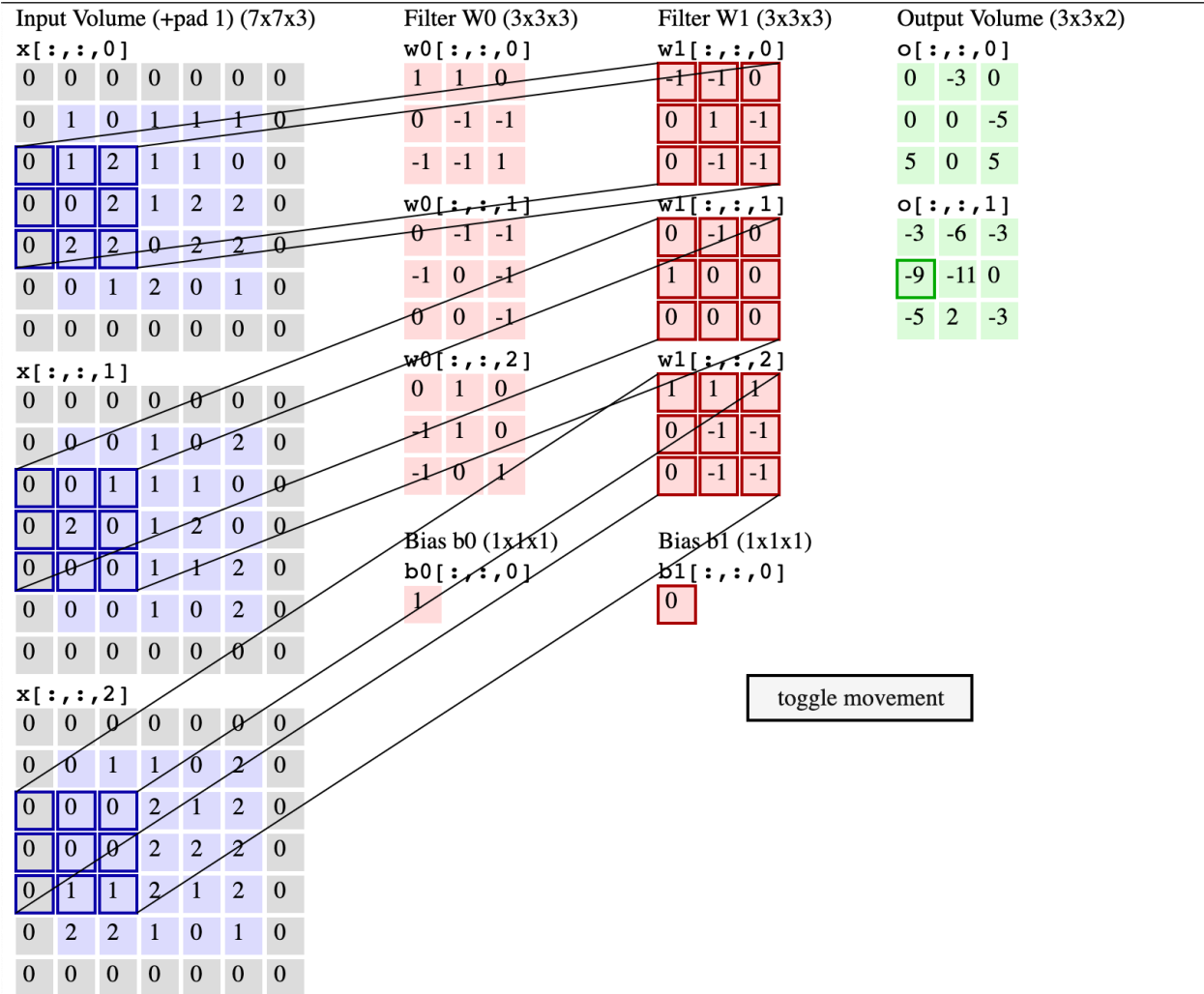
2D convolution



Statistics

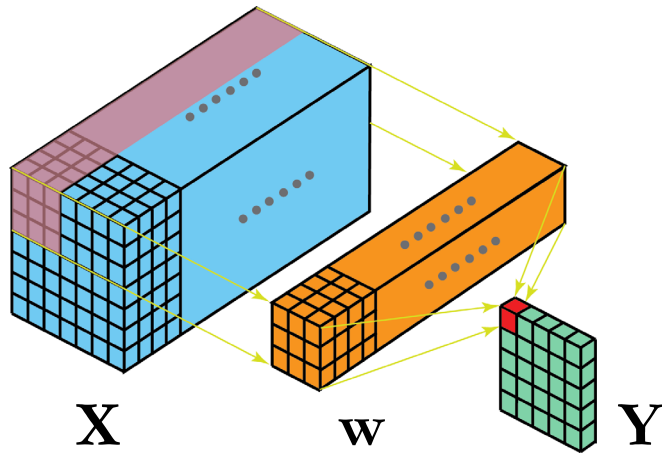
- Parameter size
 - Weights: $c_o \times (c_i k_h k_w)$
 - Bias: c_o
- Output shape
 - $(c_o, o_h, o_w) = (c_o, \left\lfloor \frac{n_h + p_h - k_h}{s_h} \right\rfloor + 1, \left\lfloor \frac{n_w + p_w - k_w}{s_w} \right\rfloor + 1)$
- Computation cost
 - $O(c_o \times c_i k_h k_w \times o_h o_w)$

2D Convolution (w/ 3D kernels and data)

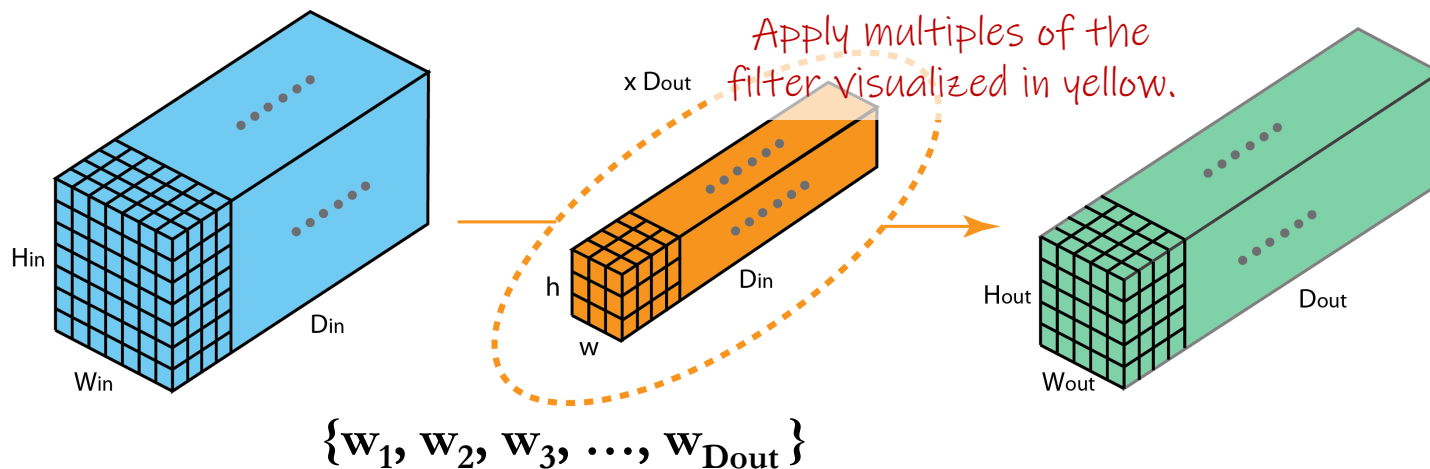


Demo [\[link\]](#)
Visualization [\[link\]](#)

2D Convolution on 3D Data

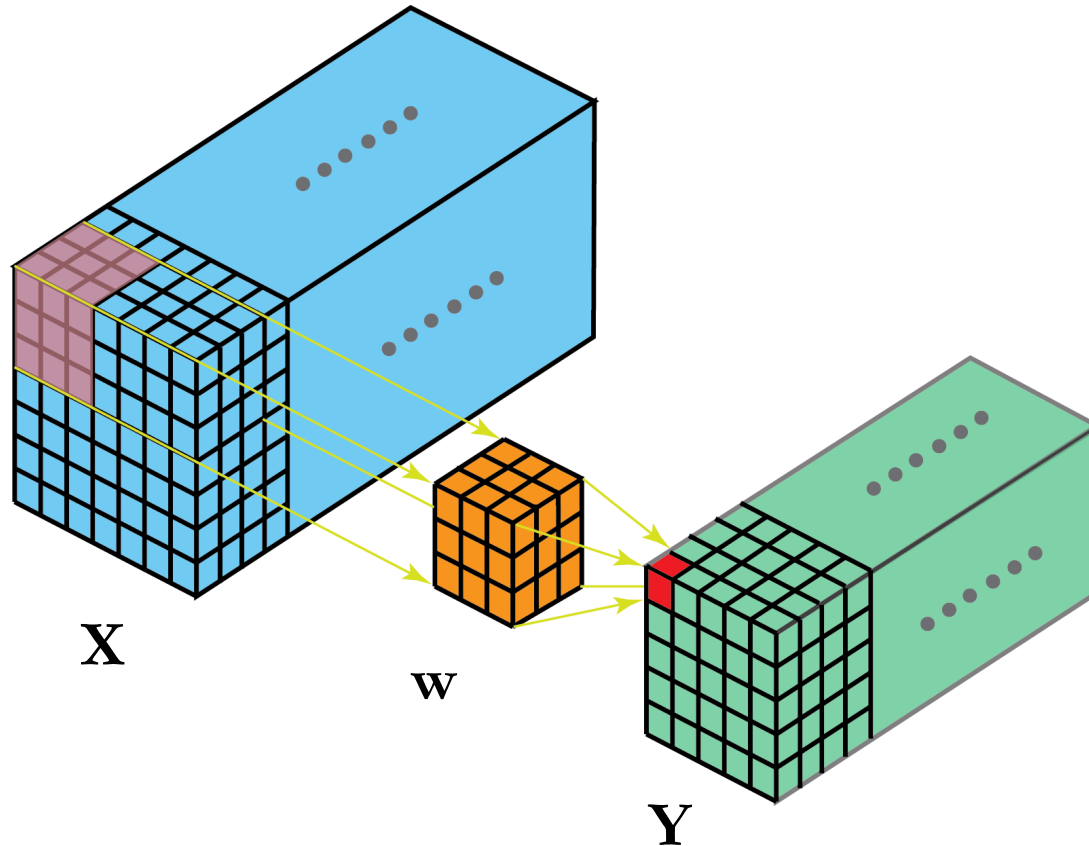


- 2D convolution slides a 3D filter matrix through the input layer.
- input layer and filter have the same depth or number of channels (equal number of channels).
- 3D filter moves only in 2 directions: height & width of the input (hence being a 2D convolution).
- output here is a one-layer matrix.”



- For multi-channelled output, apply multiple 3D filter kernels

3D Convolution on 3D Data



- 3D convolution applies a kernel (can be 1D / 2D / 3D) that moves in all 3D dimensions of the input data.
- note that the number of channels in the kernel must be less than the number of channels in the input data.

<https://towardsdatascience.com/a-comprehensive-introduction-to-different-types-of-convolutions-in-deep-learning-669281e58215>

Implementation

- Forward
 - Convert input feature maps $\mathbf{X}$ into matrix $\hat{\mathbf{X}}$ (img2col) of size $(c_i k_w k_h \times o_h o_w)$
 - Reshape the filters $\mathbf{W}$ to $(c_o \times c_i k_h k_w)$
 - $\mathbf{Y} = \mathbf{W} \hat{\mathbf{X}} + \mathbf{b}$
 - Computational cost, $O(c_o \times c_i \times k_w \times k_h \times o_h \times o_w)$
- Backward
 - Given $\frac{\partial L}{\partial \mathbf{Y}}$
 - Compute $\frac{\partial L}{\partial \mathbf{W}} = \frac{\partial L}{\partial \mathbf{Y}} \hat{\mathbf{X}}^T$, $\frac{\partial L}{\partial \hat{\mathbf{X}}} = \mathbf{W}^T \frac{\partial L}{\partial \mathbf{Y}}$
 - Column to receptive field transformation to get gradient wrt original $\mathbf{X}$